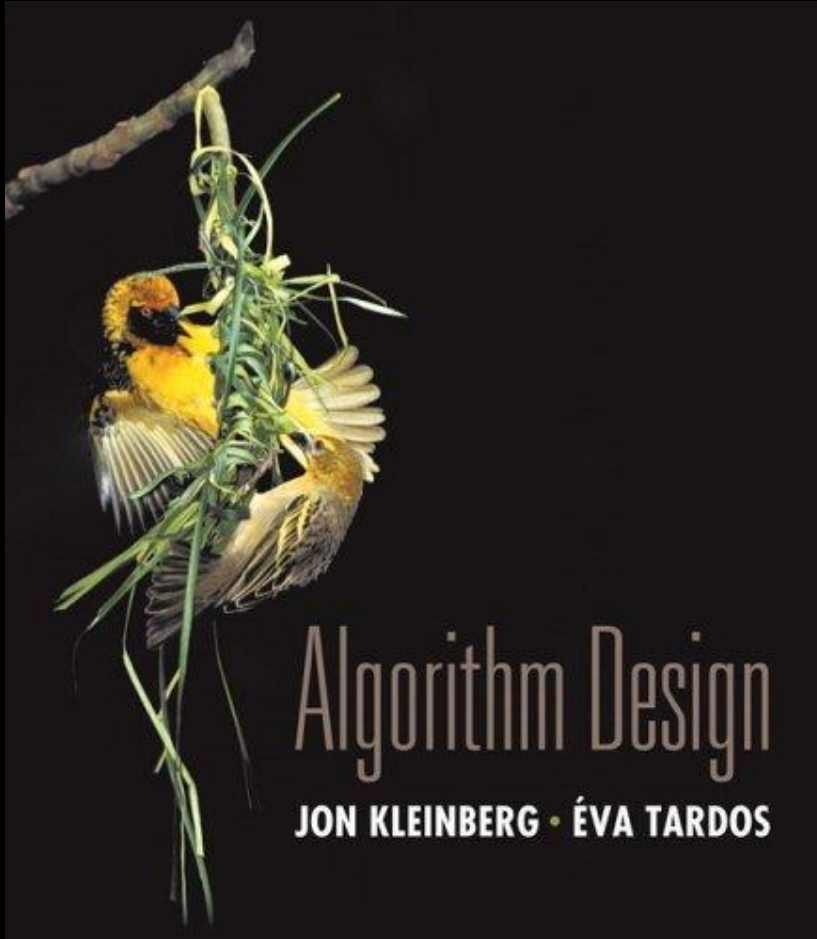


Chapter 8

NP and Computational Intractability



Slides by Kevin Wayne.
Copyright © 2005 Pearson-Addison Wesley.
All rights reserved.

8.4 NP-Completeness

Polynomial Transformation

Def. Problem X **polynomial reduces** (Cook) to problem Y if arbitrary instances of problem X can be solved using:

- Polynomial number of standard computational steps, plus
- Polynomial number of calls to oracle that solves problem Y .

Def. Problem X **polynomial transforms** (Karp) to problem Y if given any input x to X , we can construct an input y in poly-time such that x is a yes instance of X iff y is a yes instance of Y .

↑
we require $|y|$ to be of size polynomial in $|x|$

Note. Polynomial transformation is polynomial reduction with just one call to oracle for Y , exactly at the end of the algorithm for X . Almost all previous reductions were of this form.

Open question. Are these two concepts the same?

↑
we abuse notation $\leq_p$ and blur distinction

NP-Complete

NP-complete. A problem Y in NP with the property that for every problem X in NP, $X \leq_p Y$.

Theorem. Suppose Y is an NP-complete problem. Then Y is solvable in poly-time iff $P = NP$.

Pf. $\Leftarrow$ If $P = NP$ then Y can be solved in poly-time since Y is in NP.

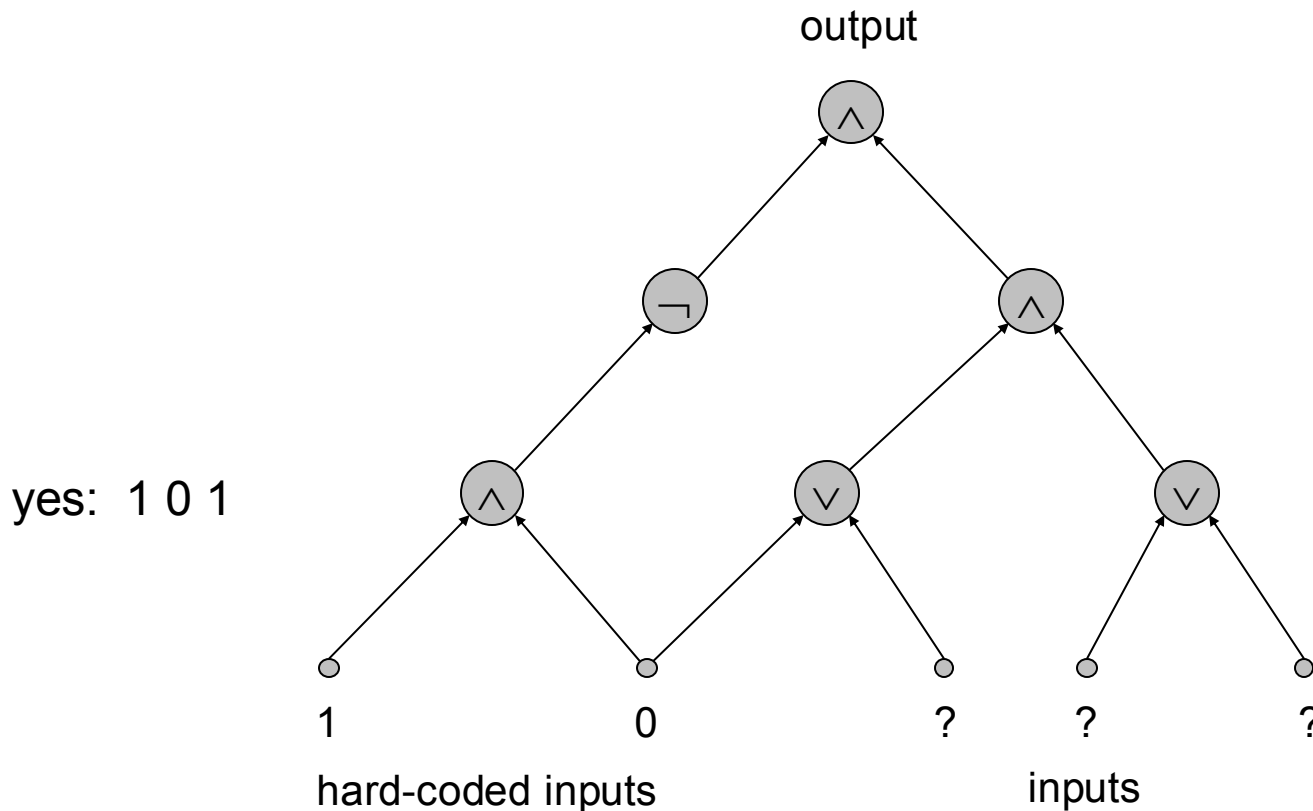
Pf. $\Rightarrow$ Suppose Y can be solved in poly-time.

- Let X be any problem in NP. Since $X \leq_p Y$, we can solve X in poly-time. This implies $NP \subseteq P$.
- We already know $P \subseteq NP$. Thus $P = NP$. ▀

Fundamental question. Do there exist "natural" NP-complete problems?

Circuit Satisfiability

CIRCUIT-SAT. Given a combinational circuit built out of AND, OR, and NOT gates, is there a way to set the circuit inputs so that the output is 1?



The "First" NP-Complete Problem

Theorem. CIRCUIT-SAT is NP-complete. [Cook 1971, Levin 1973]

Pf. (sketch)

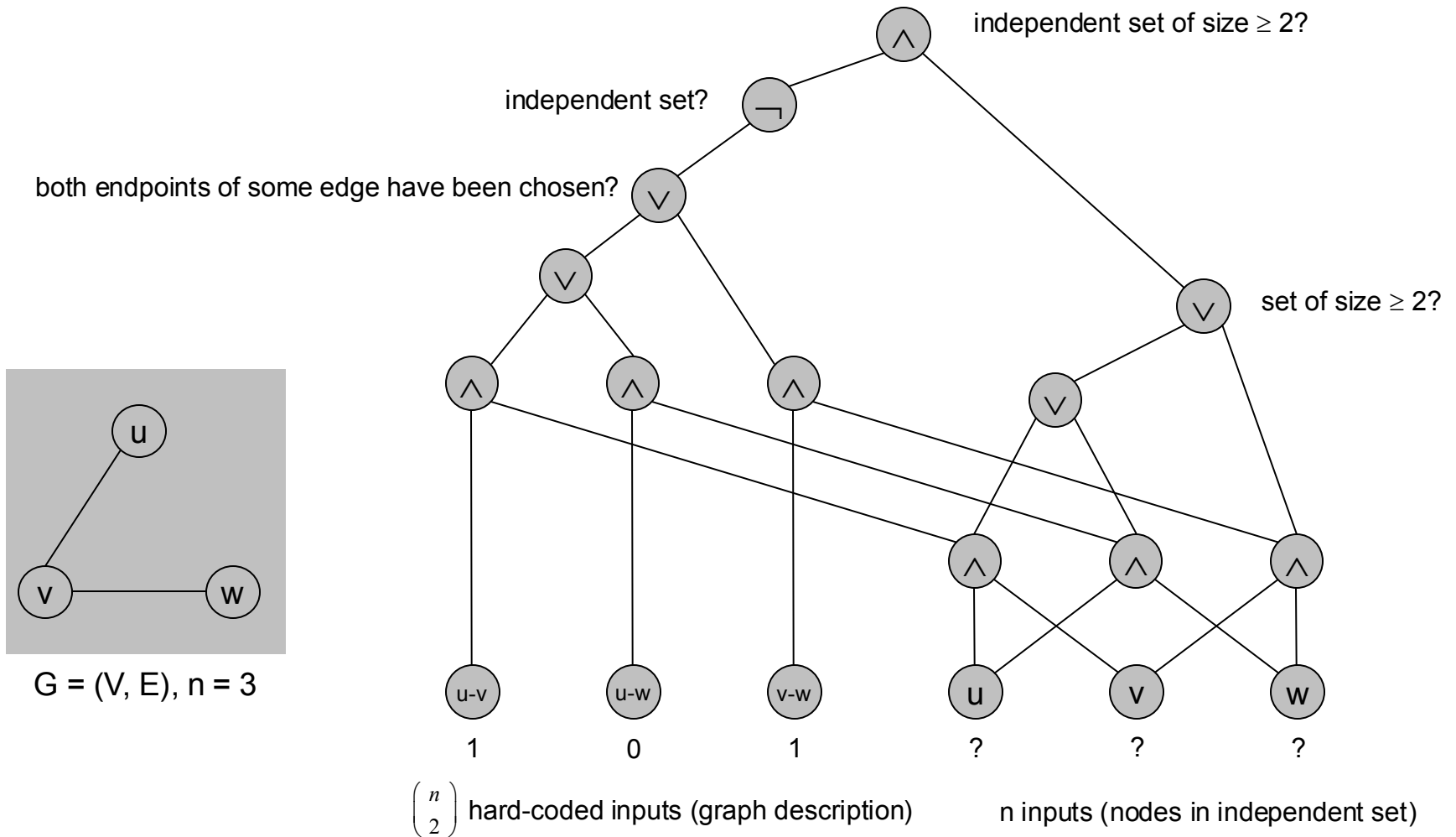
- Clearly, CIRCUIT-SAT is in NP
- Any algorithm that takes a fixed number of bits n as input and produces a yes/no answer can be represented by such a circuit. Moreover, if algorithm takes poly-time, then circuit is of poly-size.

sketchy part of proof; fixing the number of bits is important, and reflects basic distinction between algorithms and circuits

- Consider some problem X in NP. It has a poly-time certifier $C(s, t)$. To determine whether s is in X , need to know if there exists a certificate t of length $p(|s|)$ such that $C(s, t) = \text{yes}$.
- View $C(s, t)$ as an algorithm on $|s| + p(|s|)$ bits (input s , certificate t) and convert it into a poly-size circuit K .
 - first $|s|$ bits are hard-coded with s
 - remaining $p(|s|)$ bits represent (unknown) bits of t
- Circuit K is satisfiable iff $\exists t, C(s, t) = \text{yes}$.

Example

Ex. Construction below creates a circuit K whose inputs can be set so that K outputs true iff graph G has an independent set of size ≥ 2 .



Establishing NP-Completeness

Remark. Once we establish first "natural" NP-complete problem, others fall like dominoes.

Recipe to establish NP-completeness of problem Y .

- Step 1. Show that Y is in NP.
- Step 2. Choose an NP-complete problem X .
- Step 3. Prove that $X \leq_p Y$.

Theorem. If X is an NP-complete problem and Y is a problem in NP with the property that $X \leq_p Y$, then Y is NP-complete.

Pf. Let W be any problem in NP. Then $W \leq_p X \leq_p Y$.

- By transitivity, $W \leq_p Y$.
- Hence Y is NP-complete. ▀

↑ ↑
by definition of by assumption
NP-complete

3-SAT is NP-Complete

Review of 3-SAT:

- **Literal:** A Boolean variable or its negation. x_i or $\overline{x_i}$
- **Clause:** A disjunction of literals. $C_j = x_1 \vee \overline{x_2} \vee x_3$
- **Conjunctive normal form:** A propositional formula Φ that is the conjunction of clauses. $\Phi = C_1 \wedge C_2 \wedge C_3 \wedge C_4$
- **SAT:** Given CNF formula Φ , does it have a satisfying truth assignment?
- **3-SAT:** SAT where each clause contains exactly 3 literals.

Ex: $(\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_1} \vee x_2 \vee x_4)$

Yes: $x_1 = \text{true}, x_2 = \text{true}, x_3 = \text{false}, x_4 = \text{false}.$

Logical equivalence (from CS181 Artificial Intelligence)

Two sentences are **logically equivalent** iff true in the same models

$$\begin{aligned}(\alpha \wedge \beta) &\equiv (\beta \wedge \alpha) && \text{commutativity of } \wedge \\(\alpha \vee \beta) &\equiv (\beta \vee \alpha) && \text{commutativity of } \vee \\((\alpha \wedge \beta) \wedge \gamma) &\equiv (\alpha \wedge (\beta \wedge \gamma)) && \text{associativity of } \wedge \\((\alpha \vee \beta) \vee \gamma) &\equiv (\alpha \vee (\beta \vee \gamma)) && \text{associativity of } \vee \\\neg(\neg\alpha) &\equiv \alpha && \text{double-negation elimination} \\(\alpha \Rightarrow \beta) &\equiv (\neg\beta \Rightarrow \neg\alpha) && \text{contraposition} \\(\alpha \Rightarrow \beta) &\equiv (\neg\alpha \vee \beta) && \text{implication elimination} \\(\alpha \Leftrightarrow \beta) &\equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) && \text{biconditional elimination} \\\neg(\alpha \wedge \beta) &\equiv (\neg\alpha \vee \neg\beta) && \text{de Morgan} \\\neg(\alpha \vee \beta) &\equiv (\neg\alpha \wedge \neg\beta) && \text{de Morgan} \\(\alpha \wedge (\beta \vee \gamma)) &\equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) && \text{distributivity of } \wedge \text{ over } \vee \\(\alpha \vee (\beta \wedge \gamma)) &\equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) && \text{distributivity of } \vee \text{ over } \wedge\end{aligned}$$

3-SAT is NP-Complete

Theorem. 3-SAT is NP-complete.

Pf. Suffices to show that $\text{CIRCUIT-SAT} \leq_p \text{3-SAT}$ since 3-SAT is in NP.

- Let K be any circuit. We can construct an instance of 3-SAT Φ :

- Step 1: Create a 3-SAT variable x_i for each circuit element i .

- Step 2: Make circuit compute correct values at each node:

- $x_2 = \neg x_3 \Rightarrow$ add 2 clauses: $x_2 \vee x_3, \overline{x_2} \vee \overline{x_3}$

- $x_1 = x_4 \vee x_5 \Rightarrow$ add 3 clauses: $x_1 \vee \overline{x_4}, x_1 \vee \overline{x_5}, \overline{x_1} \vee x_4 \vee x_5$

- $x_0 = x_1 \wedge x_2 \Rightarrow$ add 3 clauses: $\overline{x_0} \vee x_1, \overline{x_0} \vee x_2, x_0 \vee \overline{x_1} \vee \overline{x_2}$

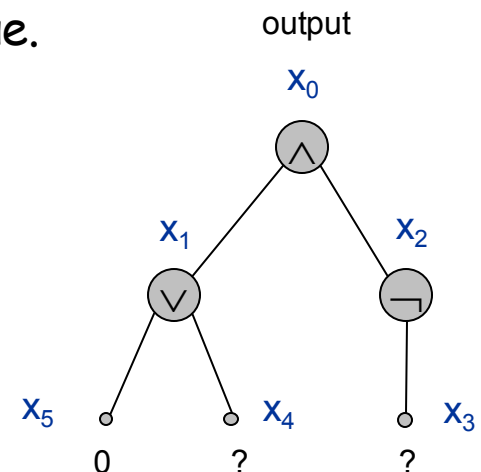
- Step 3: Hard-coded input values and output value.

- $x_5 = 0 \Rightarrow$ add 1 clause: $\overline{x_5}$

- $x_0 = 1 \Rightarrow$ add 1 clause: x_0

- Final step: turn clauses of length < 3 into clauses of length exactly 3 (see book p472)

- See next page.

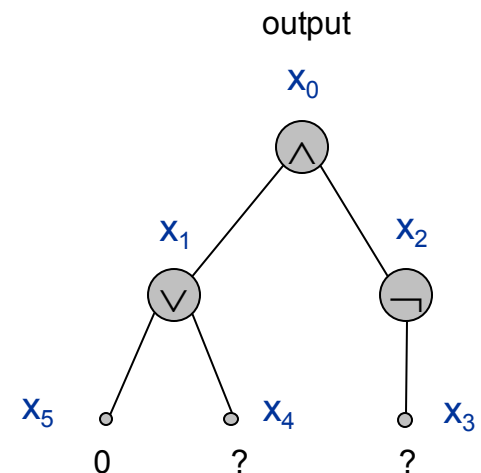


3-SAT is NP-Complete

Theorem. 3-SAT is NP-complete.

Pf. (cont.)

- Final step: turn clauses of length < 3 into clauses of length exactly 3
- By adding dummy nodes z or $z1$ and $z2$
- For the clause with length = 2
 - $(x \vee y \vee z) \wedge (x \vee y \vee \neg z)$
- For the clause with length = 1
 - $(x \vee z1 \vee z2) \wedge (x \vee z1 \vee \neg z2) \wedge (x \vee \neg z1 \vee z2) \wedge (x \vee \neg z1 \vee \neg z2).$ ■



3-SAT is NP-Complete

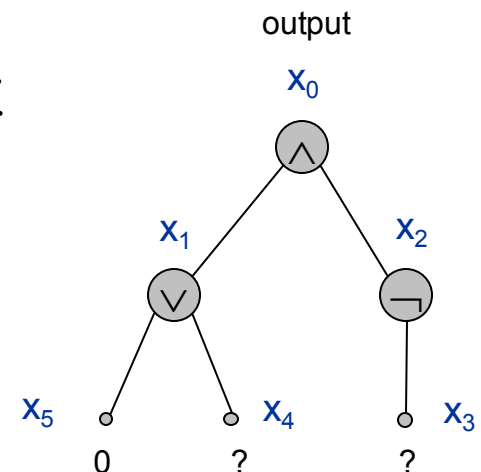
Lemma. Φ is satisfiable iff. the inputs of K can be set s.t. it outputs 1.

Pf. $\Leftarrow$

- Suppose there are inputs of K s.t. it outputs 1.
- Compute the values of all the nodes in K .
- According to our construction, this set of values satisfies Φ .

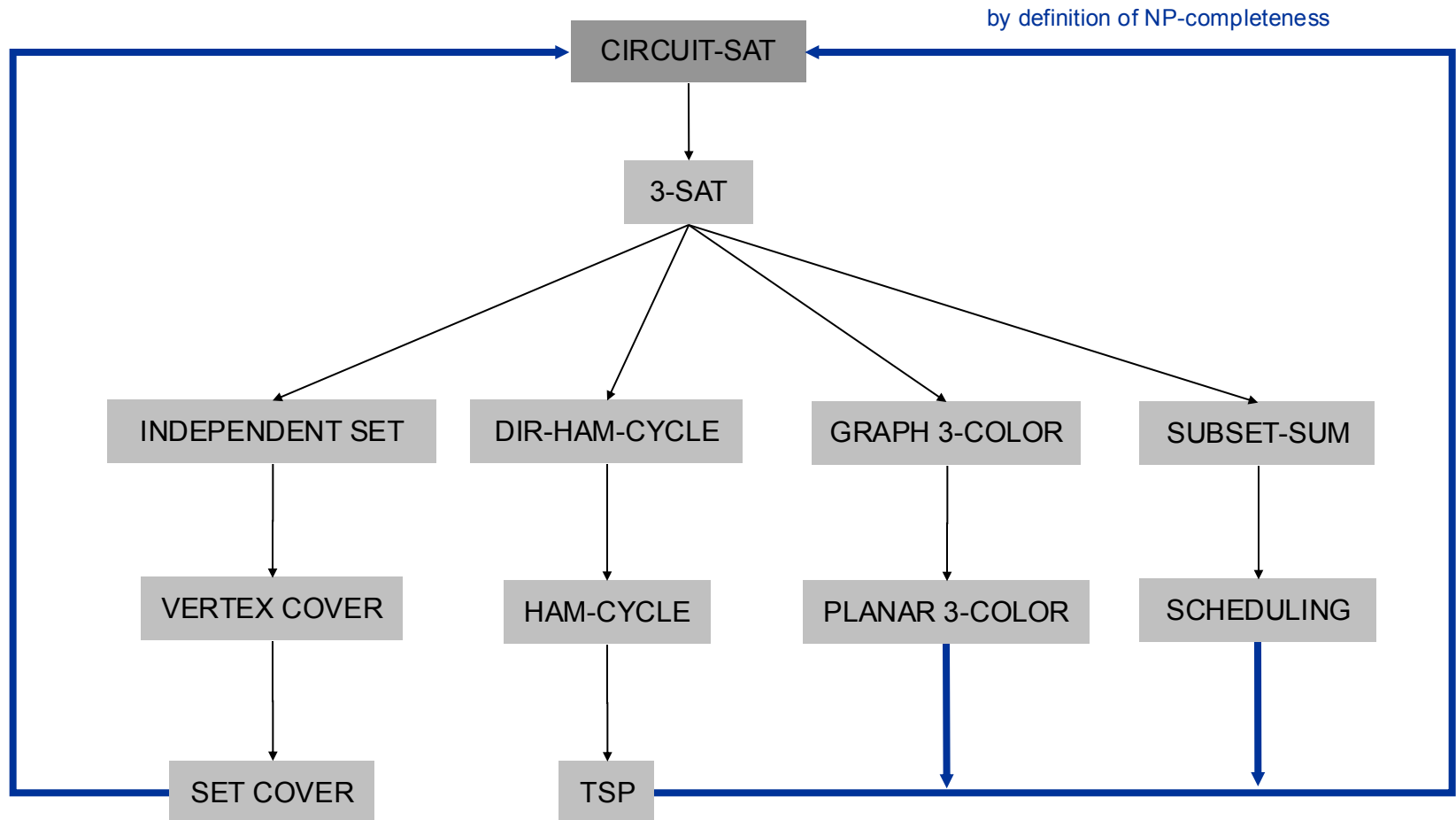
$\Rightarrow$

- Suppose Φ is satisfiable.
- The variable assignment of Φ specifies the values of all the nodes in K .
- Our construction ensures that all the nodes in K are correctly computed and the output is 1.



NP-Completeness

Observation. All problems below are NP-complete and polynomial reduce to one another!



Some NP-Complete Problems

Six basic genres of NP-complete problems and paradigmatic examples.

- Packing problems: SET-PACKING, INDEPENDENT SET.
- Covering problems: SET-COVER, VERTEX-COVER.
- Constraint satisfaction problems: SAT, 3-SAT.
- Sequencing problems: HAMILTONIAN-CYCLE, TSP.
- Partitioning problems: 3D-MATCHING, 3-COLOR.
- Numerical problems: SUBSET-SUM, KNAPSACK.

Practice. Most NP problems are known to be either in P or NP-complete.

Notable exceptions. Factoring, graph isomorphism, Nash equilibrium.

Theorem. [Ladner 1975] Unless $P = NP$, there exist problems in NP that are neither in P nor NP-complete.

Extent and Impact of NP-Completeness

Extent of NP-completeness. [Papadimitriou 1995]

- 6,000 citations per year (title, abstract, keywords).
 - more than "compiler", "operating system", "database"
- Prime intellectual export of CS to other disciplines.
- Broad applicability and classification power.
- "Captures vast domains of computational, scientific, mathematical endeavors, and seems to roughly delimit what mathematicians and scientists had been aspiring to compute feasibly."

NP-completeness can guide scientific inquiry.

- 1926: Ising introduces simple model for phase transitions.
- 1944: Onsager finds closed-form solution to 2D-Ising.
- 19xx: Feynman and other top minds seek 3D solution.
- 2000: Istrail proves 3D-Ising is NP-complete; thus search for closed-form solution to 3D-Ising appears doomed.

More Hard Computational Problems

- Aerospace engineering:** optimal mesh partitioning for finite elements.
- Biology:** protein folding.
- Chemical engineering:** heat exchanger network synthesis.
- Civil engineering:** equilibrium of urban traffic flow.
- Economics:** computation of arbitrage in financial markets with friction.
- Electrical engineering:** VLSI layout.
- Environmental engineering:** optimal placement of contaminant sensors.
- Financial engineering:** find minimum risk portfolio of given return.
- Game theory:** find Nash equilibrium that maximizes social welfare.
- Genomics:** phylogeny reconstruction.
- Mechanical engineering:** structure of turbulence in sheared flows.
- Medicine:** reconstructing 3-D shape from biplane angiocardiogram.
- Operations research:** optimal resource allocation.
- Physics:** partition function of 3-D Ising model in statistical mechanics.
- Politics:** Shapley-Shubik voting power.
- Pop culture:** Minesweeper consistency.
- Statistics:** optimal experimental design.

Terminology

NP-complete. A problem in NP such that every problem in NP polynomial reduces to it.

NP-hard. [Bell Labs, Steve Cook, Ron Rivest, Sartaj Sahni]
A problem such that every problem in NP reduces to it.



Stephen Cook

Stephen Cook

- 1971 paper "The Complexity of Theorem Proving Procedures"
 - formalized the notions of polynomial-time reduction and NP-completeness
 - proved the existence of an NP-complete problem (SAT)
- Bio
 - Joined UC Berkeley in 1966 but was denied tenure in 1970
 - Joined U of Toronto in 1970
 - 1982 Turing award



Richard Karp

"It is to our everlasting shame that we were unable to persuade the math department to give him tenure." -- Richard Karp (1985 Turing award)

8.9 co-NP and the Asymmetry of NP

Asymmetry of NP

NP. Decision problems for which there is a poly-time certifier.

Ex. SAT, HAM-CYCLE, COMPOSITES.

Asymmetry of NP. We only need to have short proofs of *yes* instances.

It may **not** be easy to show proofs of *no* instances.

Ex 1. SAT vs. TAUTOLOGY.

- Can prove a CNF formula is satisfiable by giving such an assignment.
- How could we prove that a formula is **not** satisfiable?

Ex 2. HAM-CYCLE vs. NO-HAM-CYCLE.

- Can prove a graph is Hamiltonian by giving such a Hamiltonian cycle.
- How could we prove that a graph is **not** Hamiltonian?

NP and co-NP

NP. Decision problems for which there is a poly-time certifier.

Ex. SAT, HAM-CYCLE, COMPOSITES.

Def. Given a decision problem X , its **complement** $\overline{X}$ is the same problem with the _{yes} and _{no} answers reverse.

Ex. $\overline{X} = \{ 0, 1, 4, 6, 8, 9, 10, 12, 14, 15, \dots \}$
 $X = \{ 2, 3, 5, 7, 11, 13, 17, 23, 29, \dots \}$

co-NP. Complements of decision problems in NP.

- for _{no} instance, there is a succinct disqualifier

Ex. TAUTOLOGY, NO-HAM-CYCLE, PRIMES.

NP = co-NP ?

Fundamental question. Does NP = co-NP?

- Do _{yes} instances have succinct certificates iff _{no} instances do?
- Consensus opinion: no.

Theorem. If $\text{NP} \neq \text{co-NP}$, then $\text{P} \neq \text{NP}$.

Pf idea.

- P is closed under complementation. $x \in P \Rightarrow \overline{x} \in P$
- If $\text{P} = \text{NP}$, then NP is closed under complementation.
- In other words, $\text{NP} = \text{co-NP}$.
- This is the contrapositive of the theorem.

Good Characterizations

Good characterization. [Edmonds 1965] $NP \cap co-NP$.

- If problem X is in both NP and $co-NP$, then:
 - for yes instance, there is a succinct certificate
 - for no instance, there is a succinct disqualifier
- Provides conceptual leverage for reasoning about a problem.

Ex. Given a bipartite graph, is there a perfect matching.

- If yes, can exhibit a perfect matching.
- If no, can exhibit a set of nodes S such that $|N(S)| < |S|$.

Good Characterizations

Observation. $P \subseteq NP \cap \text{co-NP}$.

Fundamental open question. Does $P = NP \cap \text{co-NP}$?

- Mixed opinions.
- Many examples where problem found to have a non-trivial good characterization, but only years later discovered to be in P.
 - linear programming [Khachiyan, 1979]
 - primality testing [Agrawal-Kayal-Saxena, 2002]

Fact. Factoring is in $NP \cap \text{co-NP}$, but not known to be in P.



if poly-time algorithm for factoring,
can break RSA cryptosystem

FACTOR is in $NP \cap co-NP$

FACTORIZE. Given an integer x , find its prime factorization.

FACTOR. Given two integers x and y , does x have a nontrivial factor less than y ?

Theorem. $FACTOR \equiv_p FACTORIZE$.

Pf. Binary search to find a factor, divide the factor and repeat

Theorem. $FACTOR$ is in $NP \cap co-NP$.

Pf.

- **Certificate:** a factor p of x that is less than y .
- **Disqualifier:** the prime factorization of x (where each prime factor is $\geq y$), along with a Pratt's certificate that each factor is prime.

Primality Testing and Factoring

Q: Is PRIMES in P?

- Yes! (Agrawal-Kayal-Saxena, 2002)

Q: Is FACTOR in P?

- Consensus: No.

RSA cryptosystem.

- Based on dichotomy between complexity of two problems.
- To use RSA, must generate large primes efficiently.
- To break RSA, must find efficient factoring algorithm.

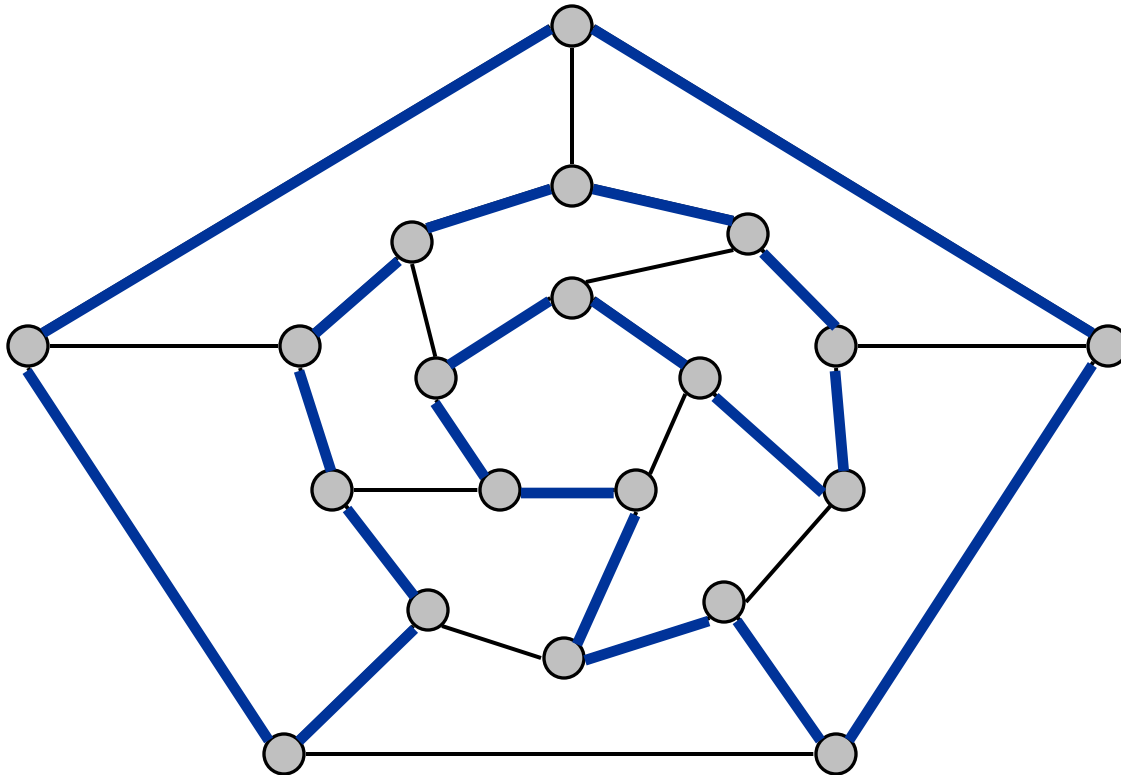
8.5 Sequencing Problems

Basic genres.

- Packing problems: SET-PACKING, INDEPENDENT SET.
- Covering problems: SET-COVER, VERTEX-COVER.
- Constraint satisfaction problems: SAT, 3-SAT.
- **Sequencing problems:** HAMILTONIAN-CYCLE, TSP.
- Partitioning problems: 3D-MATCHING, 3-COLOR.
- Numerical problems: SUBSET-SUM, KNAPSACK.

Hamiltonian Cycle

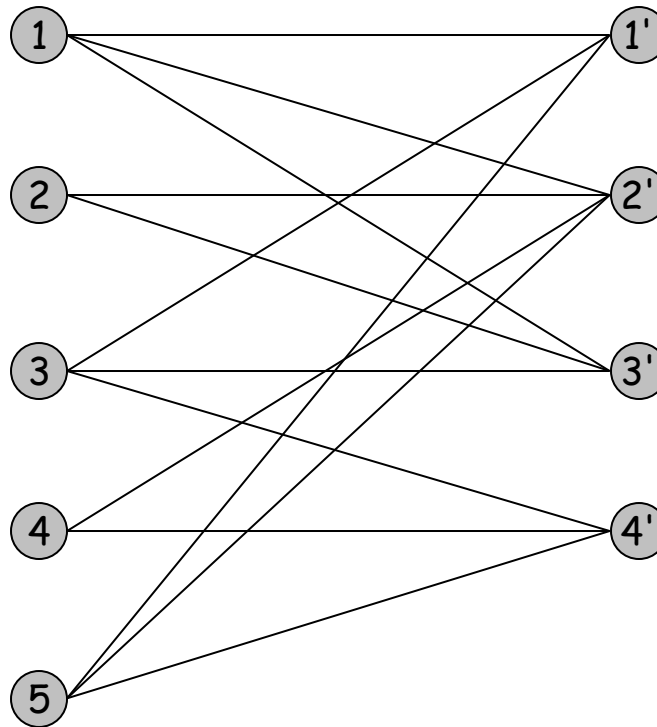
HAM-CYCLE: given an undirected graph $G = (V, E)$, does there exist a simple cycle Γ that contains every node in V .



YES: vertices and faces of a dodecahedron.

Hamiltonian Cycle

HAM-CYCLE: given an undirected graph $G = (V, E)$, does there exist a simple cycle Γ that contains every node in V .



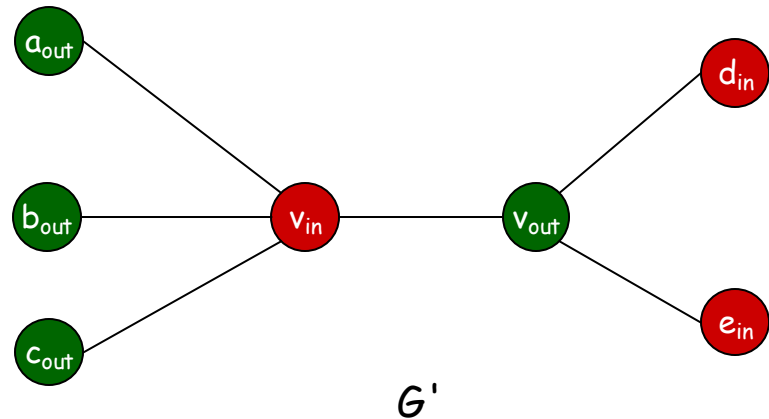
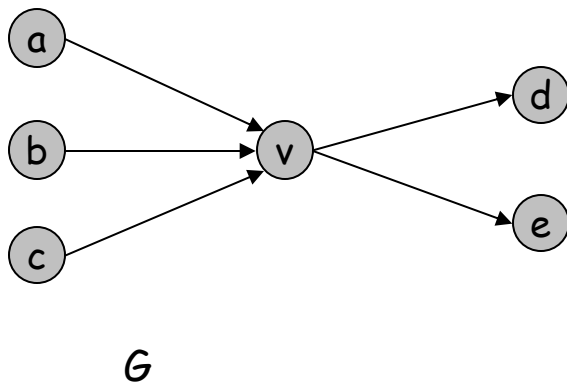
NO: bipartite graph with odd number of nodes.

Directed Hamiltonian Cycle

DIR-HAM-CYCLE: given a **digraph** $G = (V, E)$, does there exist a simple directed cycle Γ that contains every node in V ?

Claim. $\text{DIR-HAM-CYCLE} \leq_p \text{HAM-CYCLE}$.

Pf. Given a directed graph $G = (V, E)$, construct an undirected graph G'

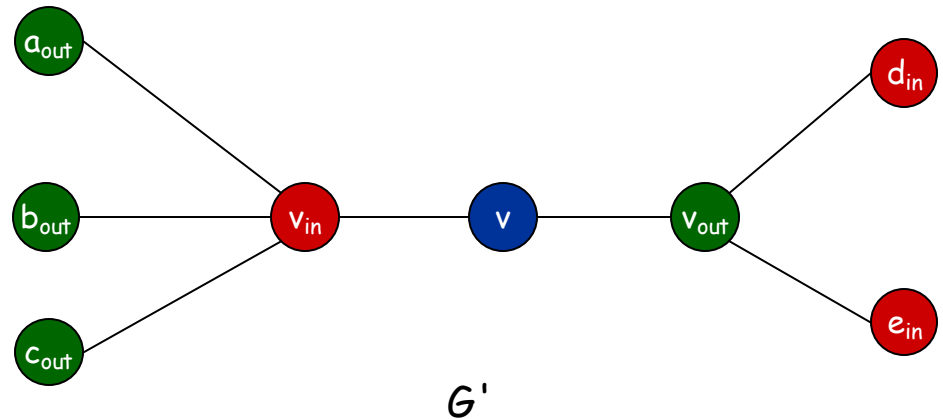
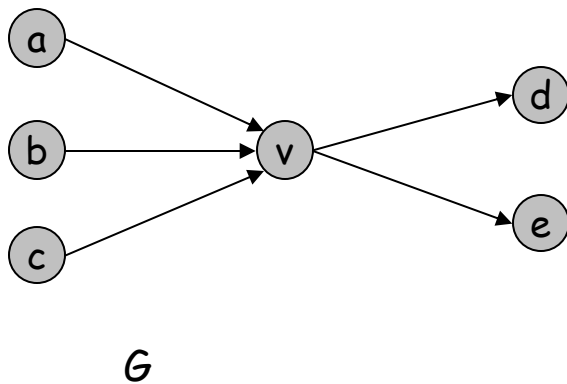


Directed Hamiltonian Cycle

DIR-HAM-CYCLE: given a **digraph** $G = (V, E)$, does there exist a simple directed cycle Γ that contains every node in V ?

Claim. $\text{DIR-HAM-CYCLE} \leq_p \text{HAM-CYCLE}$.

Pf. Given a directed graph $G = (V, E)$, construct an undirected graph G' with $3n$ nodes.



Directed Hamiltonian Cycle

Claim. G has a Hamiltonian cycle iff G' does.

Pf. $\Rightarrow$

- Suppose G has a directed Hamiltonian cycle Γ .
- Then G' has an undirected Hamiltonian cycle (same order).

Pf. $\Leftarrow$

- Suppose G' has an undirected Hamiltonian cycle Γ' .
- Γ' must visit nodes in G' using one of following two orders:
 ..., B, G, R, B, G, R, B, G, R, B, ...
 ..., B, R, G, B, R, G, B, R, G, B, ...
- Blue nodes in Γ' make up directed Hamiltonian cycle Γ in G , or reverse of one. ▫

3-SAT Reduces to Directed Hamiltonian Cycle

Claim. $3\text{-SAT} \leq_p \text{DIR-HAM-CYCLE}$.

Pf. Given an instance Φ of 3-SAT, we construct an instance of DIR-HAM-CYCLE that has a Hamiltonian cycle iff Φ is satisfiable.

Review of 3-SAT:

- **Literal:** A Boolean variable or its negation. x_i or $\overline{x_i}$
- **Clause:** A disjunction of literals. $C_j = x_1 \vee \overline{x_2} \vee x_3$
- **Conjunctive normal form:** A propositional formula Φ that is the conjunction of clauses. $\Phi = C_1 \wedge C_2 \wedge C_3 \wedge C_4$
- **SAT:** Given CNF formula Φ , does it have a satisfying truth assignment?
- **3-SAT:** SAT where each clause contains exactly 3 literals.

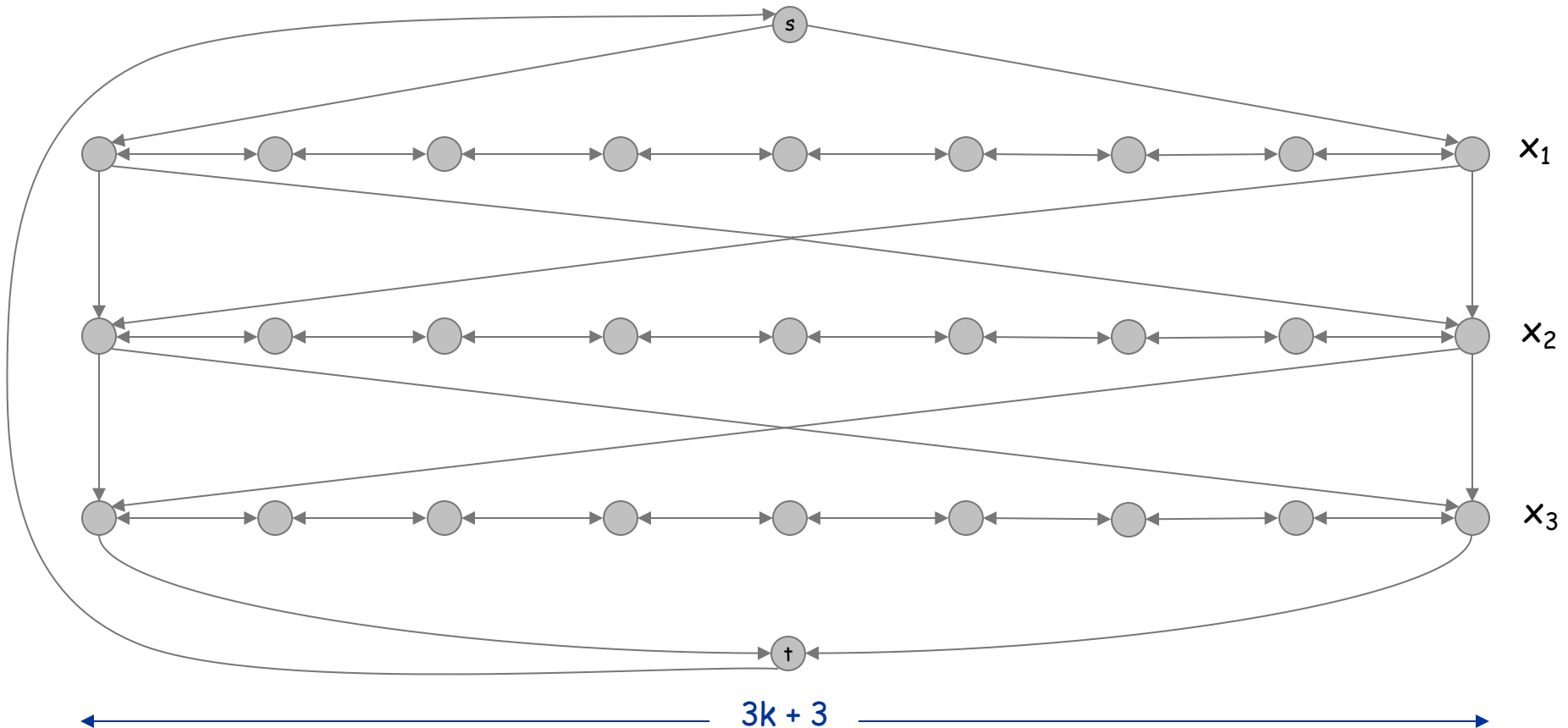
Ex: $(\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_1} \vee x_2 \vee x_4)$

Yes: $x_1 = \text{true}, x_2 = \text{true}, x_3 = \text{false}, x_4 = \text{false}$.

3-SAT Reduces to Directed Hamiltonian Cycle

Construction. Given 3-SAT instance Φ with n variables x_i and k clauses.

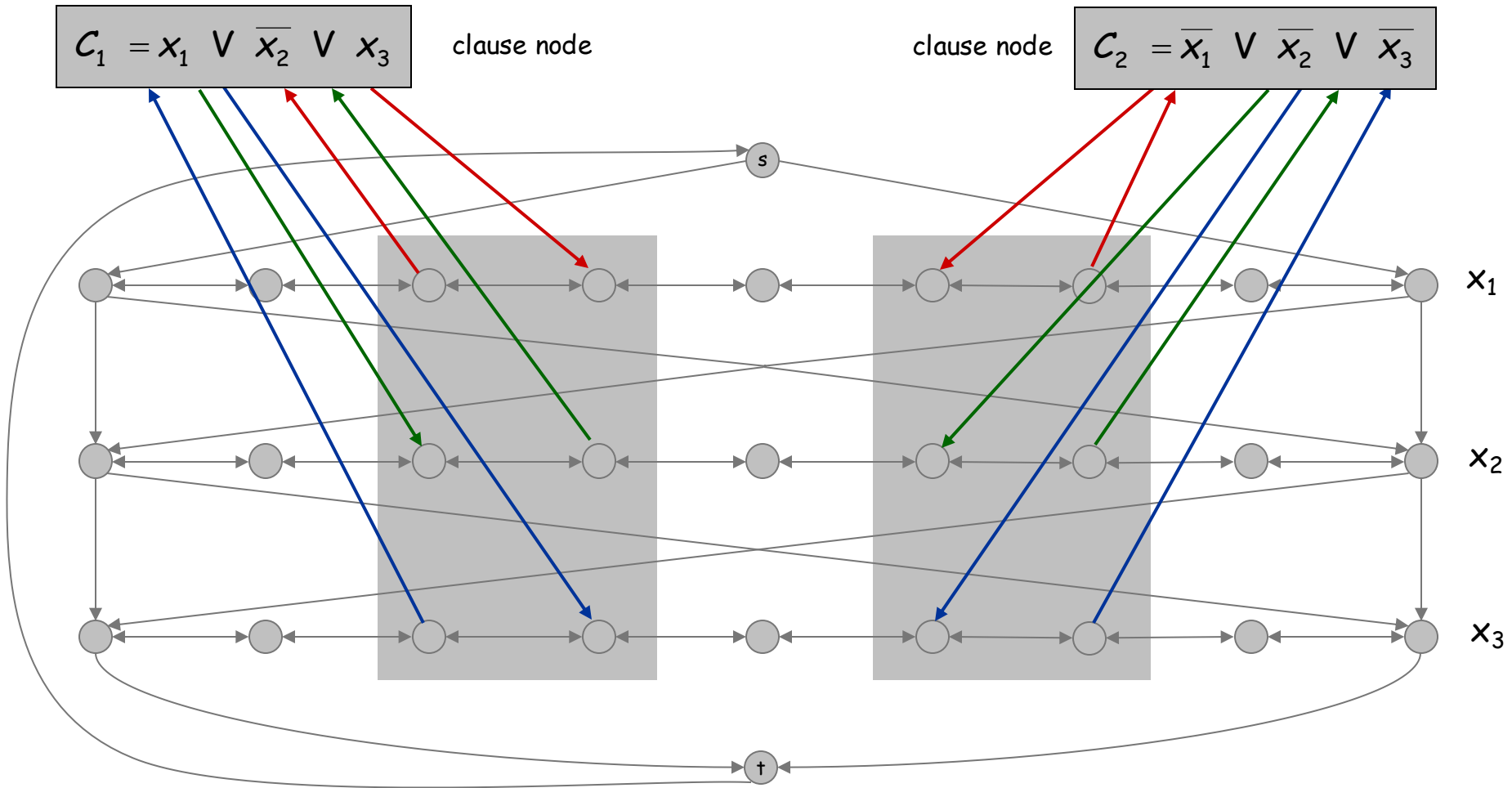
- Construct G to have 2^n Hamiltonian cycles.
- Intuition: traverse path i from left to right $\Leftrightarrow$ set variable $x_i = 1$.



3-SAT Reduces to Directed Hamiltonian Cycle

Construction. Given 3-SAT instance Φ with n variables x_i and k clauses.

- For each clause: add a node and 6 edges.



3-SAT Reduces to Directed Hamiltonian Cycle

Claim. Φ is satisfiable iff G has a Hamiltonian cycle.

Pf. $\Rightarrow$

- Suppose 3-SAT instance has satisfying assignment x^* .
- Then, define Hamiltonian cycle in G as follows:
 - if $x_i^* = 1$, traverse row i from left to right
 - if $x_i^* = 0$, traverse row i from right to left
 - for each clause C_j , there will be at least one row i in which we are going in "correct" direction to splice node C_j into tour

3-SAT Reduces to Directed Hamiltonian Cycle

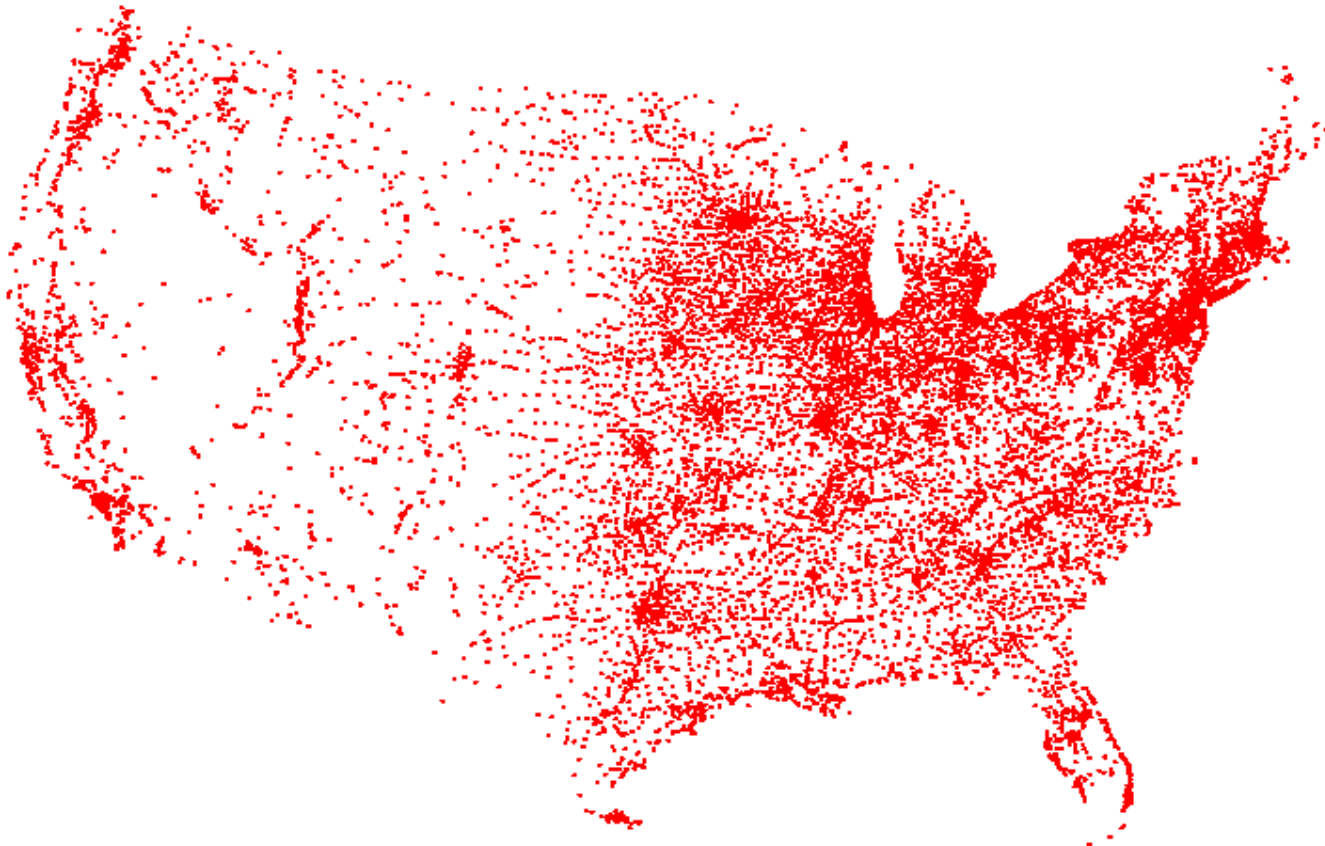
Claim. Φ is satisfiable iff G has a Hamiltonian cycle.

Pf. $\Leftarrow$

- Suppose G has a Hamiltonian cycle Γ .
- If Γ enters clause node C_j , it must depart on mate edge.
 - thus, nodes immediately before and after C_j are connected by an edge e in G
 - removing C_j from cycle, and replacing it with edge e yields Hamiltonian cycle on $G - \{C_j\}$
- Continuing in this way, we are left with Hamiltonian cycle Γ' in $G - \{C_1, C_2, \dots, C_k\}$.
- Set $x^*_i = 1$ iff Γ' traverses row i left to right.
- Since Γ visits each clause node C_j , at least one of the paths is traversed in "correct" direction, and each clause is satisfied. ▀

Traveling Salesperson Problem

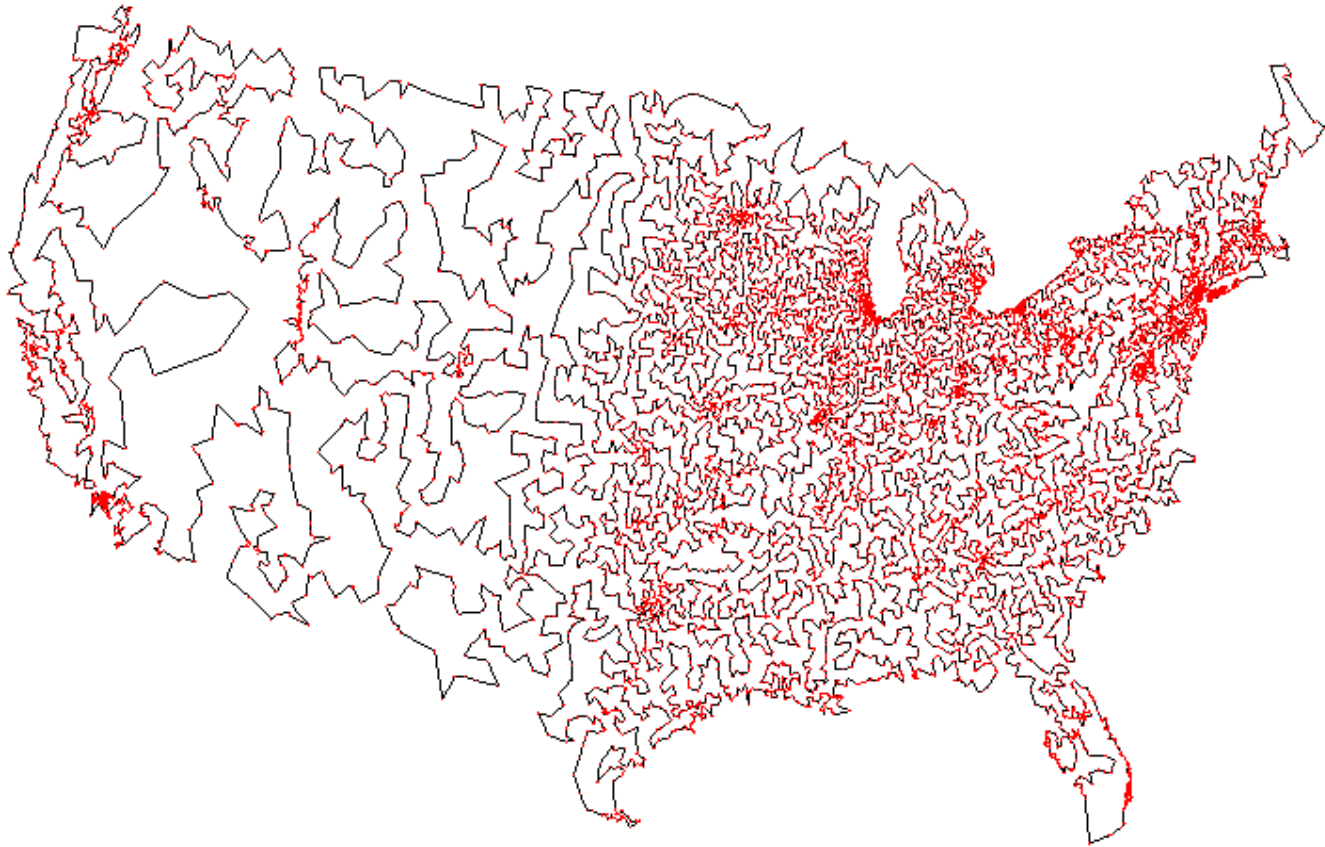
TSP. Given a set of n cities and a pairwise distance function $d(u, v)$, is there a tour of length $\leq D$?



All 13,509 cities in US with a population of at least 500
Reference: <http://www.tsp.gatech.edu>

Traveling Salesperson Problem

TSP. Given a set of n cities and a pairwise distance function $d(u, v)$, is there a tour of length $\leq D$?



Optimal TSP tour
Reference: <http://www.tsp.gatech.edu>

Traveling Salesperson Problem

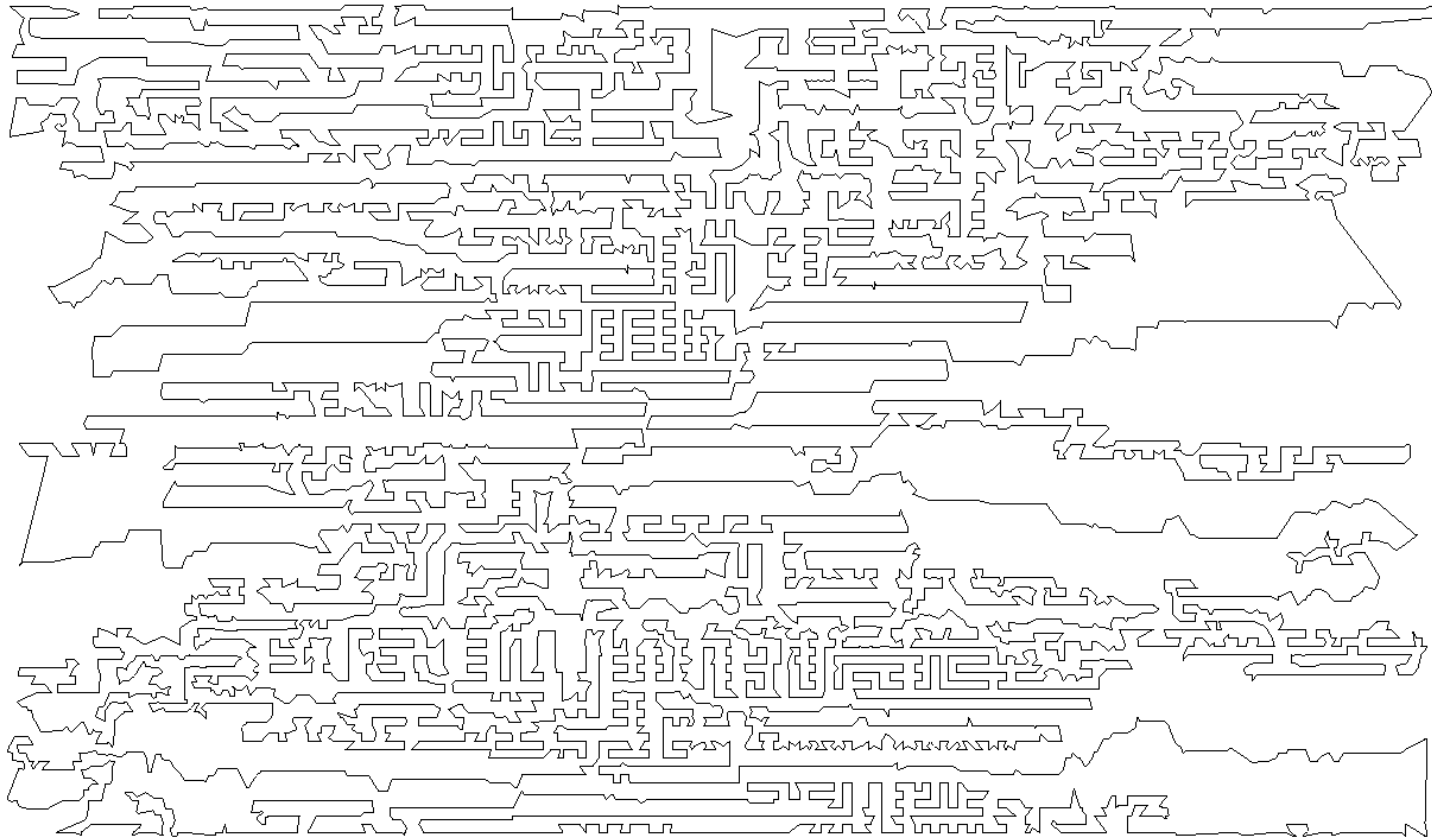
TSP. Given a set of n cities and a pairwise distance function $d(u, v)$, is there a tour of length $\leq D$?



11,849 holes to drill in a programmed logic array
Reference: <http://www.tsp.gatech.edu>

Traveling Salesperson Problem

TSP. Given a set of n cities and a pairwise distance function $d(u, v)$, is there a tour of length $\leq D$?



Optimal TSP tour
Reference: <http://www.tsp.gatech.edu>

Traveling Salesperson Problem

TSP. Given a set of n cities and a pairwise distance function $d(u, v)$, is there a tour of length $\leq D$?

HAM-CYCLE: given a graph $G = (V, E)$, does there exist a simple cycle that contains every node in V ?

Claim. $\text{HAM-CYCLE} \leq_p \text{TSP}$.

Pf.

- Given instance $G = (V, E)$ of HAM-CYCLE, create n cities with distance function

$$d(u, v) = \begin{cases} 1 & \text{if } (u, v) \in E \\ 2 & \text{if } (u, v) \notin E \end{cases}$$

- TSP instance has tour of length $\leq n$ iff G is Hamiltonian. ▀

Longest Path

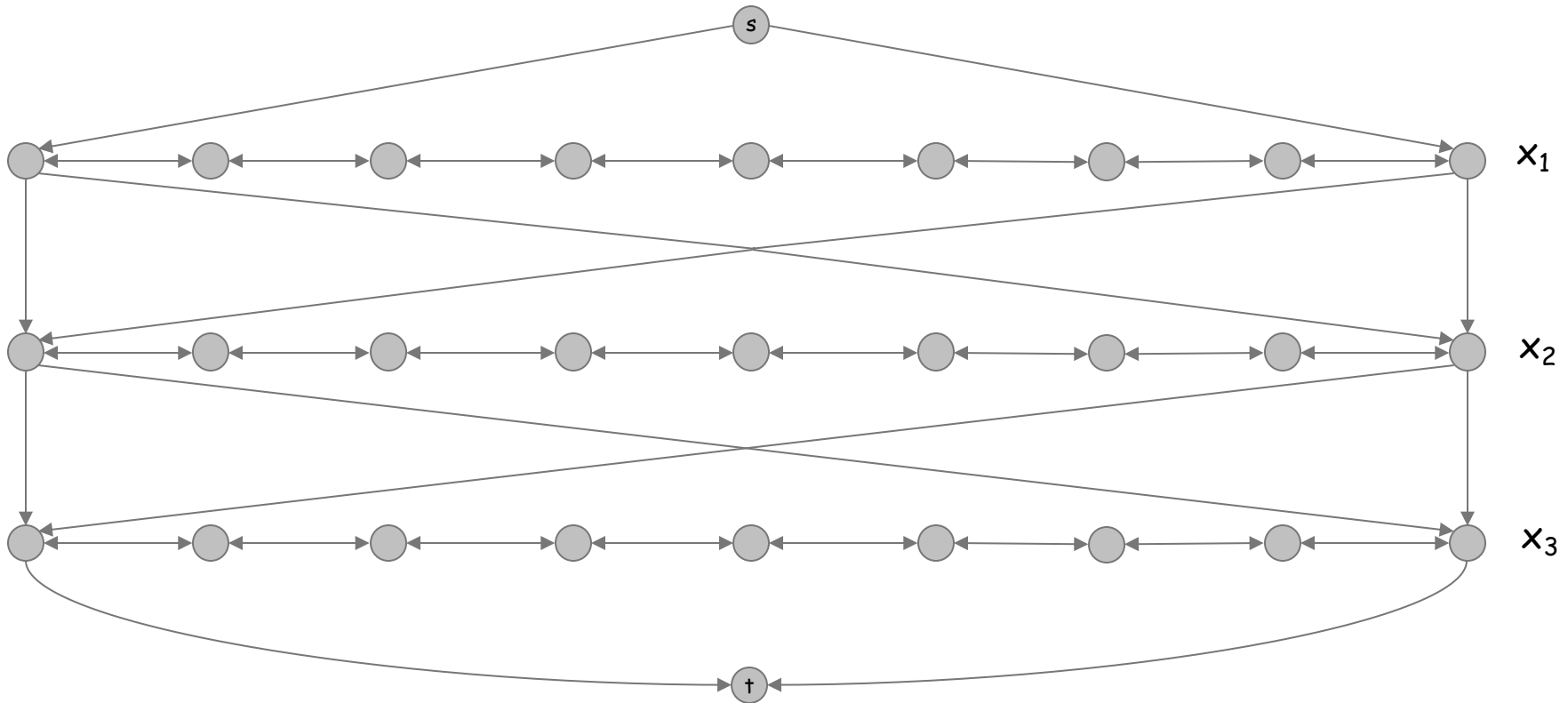
SHORTEST-PATH. Given a digraph $G = (V, E)$, does there exist a simple path of length **at most** k edges?

LONGEST-PATH. Given a digraph $G = (V, E)$, does there exist a simple path of length **at least** k edges?

Longest Path

Claim. $3\text{-SAT} \leq_p \text{LONGEST-PATH}$.

Pf 1. Redo proof for DIR-HAM-CYCLE, ignoring back-edge from t to s .



Longest Path

Claim. $3\text{-SAT} \leq_p \text{LONGEST-PATH}$.

Pf 2. Show $\text{DIR-HAM-CYCLE} \leq_p \text{LONGEST-PATH}$.

- Given an instance of DIR-HAM-CYCLE , split an arbitrary node v of the digraph into two nodes v' and v'' ; connect all the incoming edges to v' and all the outgoing edges to v'' .

